

ANSIBLE CORE MODULES

Essential Modules for Daily
System Administration

Professional Reference Documentation

Version 1.0

Table of Contents

1. Introduction
2. The package Module (yum / apt / dnf)
3. The service and systemd Modules
4. The user Module
5. The file Module
6. The copy Module
7. The lineinfile Module
8. Combining Modules: A Complete Playbook
9. Best Practices
10. Quick Reference

1. Introduction

Ansible ships with hundreds of modules, but in day-to-day system administration a small set of well-known modules does the vast majority of the work. Mastering these modules is the fastest path to writing reliable, idempotent playbooks.

This document covers six essential module families:

- **package** — a generic abstraction over the underlying platform package manager (yum, dnf, apt, zypper, pacman).
- **service** and **systemd** — manage system services and systemd units.
- **user** — create, modify, and remove local user accounts.
- **file** — manage files, directories, and symbolic links, plus ownership and permissions.
- **copy** — transfer files from the control node to managed nodes.
- **lineinfile** — ensure a specific line exists (or does not exist) in a text file.

All examples assume a project-local inventory referenced from `ansible.cfg` and a playbook file `site.yml`. Every module uses the fully qualified collection name (FQCN) form, such as `ansible.builtin.package`, which is the recommended convention.

2. The package Module (yum / apt / dnf)

The package module is a generic abstraction that automatically selects the correct backend for the target operating system. This allows a single task to work on Debian, Red Hat, SUSE, and Arch-based systems without branching logic.

2.1 When to Use package vs. Platform-Specific Modules

- `ansible.builtin.package` — preferred for cross-platform playbooks and simple installs.
- `ansible.builtin.apt` — use when you need Debian-specific features such as `update_cache` or `autoremove`.
- `ansible.builtin.yum / dnf` — use for Red Hat-family features such as `enablerepo`, `disablerepo`, or `security updates`.

2.2 Common Task Examples

Install a single package:

```
- name: Install nginx
  ansible.builtin.package:
    name: nginx
    state: present
```

Install multiple packages in one call:

```
- name: Install common utilities
  ansible.builtin.package:
    name:
      - git
      - curl
      - vim
      - htop
    state: present
```

Install a specific version:

```
- name: Install a pinned version of nginx
  ansible.builtin.package:
    name: nginx-1.24.0
    state: present
```

Install the latest version (always upgrade):

```
- name: Ensure nginx is at the latest version
  ansible.builtin.package:
    name: nginx
    state: latest
```

Remove a package:

```
- name: Remove telnet
  ansible.builtin.package:
    name: telnet
    state: absent
```

Refresh the package cache (Debian family):

```
- name: Update apt cache
  ansible.builtin.apt:
    update_cache: true
    cache_valid_time: 3600    # skip if updated within the last hour
```

2.3 Key Parameters

Parameter	Description
name	Package name, a list of names, or a wildcard. Required.
state	present, absent, latest, build-dep (apt). Default: present.
update_cache	(apt) Refresh the package index before installing.
cache_valid_time	(apt) Skip update_cache if the cache is newer than this many seconds.
autoremove	(apt) Remove unused dependencies after the operation.
enablerepo / disablerepo	(yum / dnf) Enable or disable repositories for this task only.
security	(yum / dnf) Apply only security-related updates.

Table 1 — Common package module parameters.

3. The service and systemd Modules

The `service` module manages system services across all platforms. On systemd-based hosts it delegates to `systemd`, and on SysV hosts it uses the `init` script. The `systemd` module is a more explicit variant that exposes systemd-specific features.

3.1 service vs. systemd

- `ansible.builtin.service` — preferred for portable playbooks; works with any init system.
- `ansible.builtin.systemd_service` — use for daemon-reload, masked units, or systemd-specific timer control.

In practice, most day-to-day tasks use `service`. Reserve `systemd_service` for cases where `systemd` features are required.

3.2 Common Task Examples

Start a service and enable it at boot:

```
- name: Ensure nginx is running and enabled
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: true
```

Stop a service and disable it:

```
- name: Stop and disable telnet
  ansible.builtin.service:
    name: telnet
    state: stopped
    enabled: false
```

Restart a service unconditionally:

```
- name: Restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

Reload a service (graceful, reads config without full restart):

```
- name: Reload nginx configuration
  ansible.builtin.service:
    name: nginx
    state: reloaded
```

Trigger a systemd daemon-reload after a unit file change:

```
- name: Create custom systemd unit
  ansible.builtin.copy:
    src: myapp.service
    dest: /etc/systemd/system/myapp.service
    notify: Reload systemd and restart myapp

# in handlers:
- name: Reload systemd and restart myapp
  ansible.builtin.systemd_service:
    name: myapp
    state: restarted
    daemon_reload: true
```

3.3 Service States Explained

State	Effect
started	Start if not running; no change if already running.
stopped	Stop if running; no change if already stopped.
restarted	Always stop and start — not idempotent in the strict sense.
reloaded	Reload configuration if the service supports it.

Table 2 — Service state values.

Tip: Prefer `started` with `enabled: true` in normal playbooks. Use `restarted` only inside a handler, so it runs solely when a configuration file actually changes.

4. The user Module

The user module creates, modifies, and removes local user accounts. It also manages group membership, SSH authorized keys, home directories, and account expiry.

4.1 Common Task Examples

Create a user with a specific shell and groups:

```
- name: Create the deploy user
  ansible.builtin.user:
    name: deploy
    shell: /bin/bash
    groups:
      - sudo
      - docker
    append: true           # add to groups without removing existing ones
    create_home: true
    state: present
```

Set a password (must be a pre-hashed value):

```
- name: Create user with a hashed password
  ansible.builtin.user:
    name: alice
    password: "{{ 'SecretPass123' | password_hash('sha512') }}"
    update_password: on_create # do not overwrite an existing password
    state: present
```

Authorize an SSH public key for the user:

```
- name: Install the deploy user's SSH key
  ansible.builtin.user:
    name: deploy
    authorized_key: "{{ lookup('file', 'files/deploy_id_ed25519.pub') }}"
    state: present
```

Alternative — use the dedicated `authorized_key` module:

```
- name: Add an SSH key
  ansible.posix.authorized_key:
    user: deploy
    state: present
    key: "{{ lookup('file', 'files/deploy_id_ed25519.pub') }}"
```

Remove a user and their home directory:

```
- name: Remove the temporary user
  ansible.builtin.user:
    name: tempuser
    state: absent
    remove: true # delete the home directory as well
    force: true # remove even if the user is logged in
```

4.2 Key Parameters

Parameter	Description
name	Username. Required.
state	present (default) or absent.
uid / group	Explicit user ID and primary group.
groups / append	Supplementary groups; append: true to add without removing.
shell	Login shell, e.g. <code>/bin/bash</code> or <code>/sbin/nologin</code> .
password	Pre-hashed password. Use the <code>password_hash</code> filter to generate it.
update_password	always (default) or <code>on_create</code> .
create_home	Whether to create the home directory.
authorized_key	Public key to install in <code>~/.ssh/authorized_keys</code> .
remove	Remove home directory when state: absent.
force	Force removal even if the user is logged in.

Table 3 — Common user module parameters.

5. The file Module

The `file` module manages the state of files, directories, and symbolic links. It is one of the most frequently used modules because it controls ownership, permissions, and existence.

5.1 Common Task Examples

Create a directory:

```
- name: Ensure application directory exists
  ansible.builtin.file:
    path: /opt/myapp
    state: directory
    owner: deploy
    group: deploy
    mode: '0755'
```

Recursively set ownership and permissions:

```
- name: Fix ownership across an entire tree
  ansible.builtin.file:
    path: /var/www
    state: directory
    owner: www-data
    group: www-data
    mode: '0755'
    recurse: true
```

Create an empty file:

```
- name: Touch a file
  ansible.builtin.file:
    path: /etc/myapp/initialized
    state: touch
    owner: root
    group: root
    mode: '0644'
```

Create a symbolic link:

```
- name: Symlink the current release
  ansible.builtin.file:
    src: /opt/myapp/releases/v1.2.3
    dest: /opt/myapp/current
    state: link
    force: true    # replace the link if it already exists
```

Remove a file or directory:

```
- name: Remove the old build directory
  ansible.builtin.file:
    path: /opt/myapp/build
    state: absent
```

5.2 State Values

state	Meaning
file	The path exists as a regular file; do not modify content.
directory	The path exists as a directory; create parents as needed.
touch	Create an empty file, or update the timestamp if it exists.
link	Create a symbolic link (soft link).
hard	Create a hard link.
absent	Remove the file, directory, or link recursively.

Table 4 — file module state values.

Note on mode: Always quote octal modes as strings, e.g. `mode: '0644'`. Writing `mode: 0644` without quotes causes YAML to interpret the value as octal, which is usually the same, but the quoted form is unambiguous and safer.

6. The copy Module

The copy module transfers files from the control node to managed nodes. It also allows you to write short, inline content directly, set ownership and permissions, and validate files before they are moved into place.

6.1 Common Task Examples

Copy a file from the control node:

```
- name: Deploy the application config file
  ansible.builtin.copy:
    src: files/app.conf
    dest: /etc/myapp/app.conf
    owner: root
    group: root
    mode: '0644'
```

Write inline content directly:

```
- name: Write a MOTD banner
  ansible.builtin.copy:
    content: |
      Welcome to {{ inventory_hostname }}
      Authorized access only.
    dest: /etc/motd
    owner: root
    group: root
    mode: '0644'
```

Copy an entire directory tree:

```
- name: Copy the web root
  ansible.builtin.copy:
    src: files/www/           # trailing slash copies contents, not the directory
    dest: /var/www/html/
    owner: www-data
    group: www-data
    mode: '0644'
    directory_mode: '0755'
```

Back up an existing file before overwriting:

```
- name: Update nginx.conf with backup
  ansible.builtin.copy:
    src: files/nginx.conf
    dest: /etc/nginx/nginx.conf
    backup: true
    owner: root
    group: root
    mode: '0644'
    notify: Reload nginx
```

6.2 copy vs. template

- `copy` — for static files with no variable interpolation inside the file content.
- `template` — for files that contain Jinja2 expressions, such as `{{ variable }}` or `{% for %}` loops.

If your file needs to change per host, use `template`. Reserve `copy` for truly static content.

7. The lineinfile Module

The `lineinfile` module ensures that a specific line is present in (or absent from) a text file. It is the safest way to make small, targeted edits to configuration files that are otherwise managed by the operating system or a third-party package.

7.1 Why Not Just Use `sed` or `echo`?

- `lineinfile` is idempotent: it only makes changes when the line is missing or different.
- It validates syntax before writing when `validate` is provided.
- It preserves the rest of the file and does not append duplicate lines on repeated runs.
- It supports regex matching, so it can replace a line that may have changed elsewhere in the file.

7.2 Common Task Examples

Ensure a line exists (append if missing):

```
- name: Disable password authentication for SSH
  ansible.builtin.lineinfile:
    path: /etc/ssh/sshd_config
    line: "PasswordAuthentication no"
    state: present
  notify: Restart sshd
```

Replace an existing line using a regexp:

```
- name: Set the SSH port
  ansible.builtin.lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?Port\s+'
    line: "Port 2222"
    state: present
  notify: Restart sshd
```

Add a line after a specific pattern:

```
- name: Enable UFW logging
  ansible.builtin.lineinfile:
    path: /etc/ufw/ufw.conf
    insertafter: '^# Set to yes'
    line: "LOGLEVEL=low"
```

Remove a line entirely:

```
- name: Remove a deprecated configuration line
  ansible.builtin.lineinfile:
    path: /etc/myapp/app.conf
    regexp: '^legacy_mode='
    state: absent
```

Validate a file before writing (sshd example):

```
- name: Harden SSH configuration safely
  ansible.builtin.lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^PermitRootLogin'
    line: "PermitRootLogin no"
    validate: '/usr/sbin/sshd -t -f %s'
    notify: Restart sshd
```

The validate parameter is important. It runs the given command against the temporary file before moving it into place. If the command exits non-zero, the original file is left untouched — a critical safety net when editing files like `/etc/ssh/sshd_config`.

7.3 Key Parameters

Parameter	Description
path	File to modify. Required.
line	Line to insert or replace.
regexp	Regular expression matching the line to replace or remove.
state	present (default) or absent.
insertafter	Insert after the last match of this regex, or EOF.
insertbefore	Insert before the first match of this regex, or BOF.
create	Create the file if it does not exist.
backrefs	Use regex capture groups in the replacement line; disables append-on-miss.
validate	Command run against the temp file before replacement (%s is the temp path).
backup	Save a timestamped backup of the original file.

Table 5 — Common lineinfile parameters.

8. Combining Modules: A Complete Playbook

The following playbook ties together all six modules into a realistic scenario: provisioning a small web server.

```
---
- name: Provision a web server
  hosts: webservers
  become: true

  vars:
    app_user: deploy
    app_root: /opt/myapp
    app_packages:
      - nginx
      - git
      - curl
      - ufw

  tasks:
    # ----- 1. package -----
    - name: Install required packages
      ansible.builtin.package:
        name: "{{ item }}"
        state: present
        loop: "{{ app_packages }}"

    # ----- 2. user -----
    - name: Create the application user
      ansible.builtin.user:
        name: "{{ app_user }}"
        shell: /bin/bash
        groups: sudo
        append: true
        create_home: true
        state: present

    - name: Install the deploy SSH key
      ansible.posix.authorized_key:
        user: "{{ app_user }}"
```

```

    state: present
    key: "{{ lookup('file', 'files/' + app_user + '_id_ed25519.pub') }}"

# ----- 3. file -----
- name: Create application directories
  ansible.builtin.file:
    path: "{{ item }}"
    state: directory
    owner: "{{ app_user }}"
    group: "{{ app_user }}"
    mode: '0755'
  loop:
    - "{{ app_root }}"
    - "{{ app_root }}/releases"
    - "{{ app_root }}/shared"

# ----- 4. copy -----
- name: Deploy the application configuration
  ansible.builtin.copy:
    src: files/app.conf
    dest: "{{ app_root }}/shared/app.conf"
    owner: "{{ app_user }}"
    group: "{{ app_user }}"
    mode: '0640'
    backup: true

# ----- 5. lineinfile -----
- name: Harden SSH configuration
  ansible.builtin.lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?PermitRootLogin'
    line: "PermitRootLogin no"
    validate: '/usr/sbin/sshd -t -f %s'
  notify: Restart sshd

```

```

- name: Ensure password authentication is disabled
  ansible.builtin.lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?PasswordAuthentication'
    line: "PasswordAuthentication no"
    validate: '/usr/sbin/sshd -t -f %s'
  notify: Restart sshd

# ----- 6. service -----
- name: Ensure nginx is enabled and running
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: true

handlers:
- name: Restart sshd
  ansible.builtin.service:
    name: sshd
    state: restarted

```

Observe the use of a handler to restart sshd: it runs once at the end of the play, even though two separate tasks notified it. This is the correct pattern — never restart a service after every change inside the task list.

9. Best Practices

- **Use FQCN.** Write `ansible.builtin.package` instead of `package`.
- **Prefer package over yum / apt.** Use the platform-specific modules only when you need a platform-specific feature.
- **Idempotent states only.** Use `state: present` and `state: started`, not `restarted`, outside of handlers.
- **Use handlers for restarts.** Notify a handler when a config file changes; do not restart on every run.
- **Quote octal modes.** Write `mode: '0644'`, not `mode: 0644`.
- **Use template for variable content.** Reserve copy for files that are identical on every host.
- **Always pass validate to lineinfile.** Especially for SSH and sudo configuration files.
- **Never hard-code paths.** Use variables such as `{{ app_root }}` so playbooks can be reused across environments.
- **Guard password changes.** Use `update_password: on_create` to avoid resetting passwords on every run.
- **Test with --check and --diff.** Verify what a playbook will change before applying it to production.

10. Quick Reference

Goal	Task snippet
Install a package	<code>package: name=nginx state=present</code>
Remove a package	<code>package: name=telnet state=absent</code>
Enable and start a service	<code>service: name=nginx enabled=true state=started</code>
Create a user	<code>user: name=deploy shell=/bin/bash state=present</code>
Remove a user and home	<code>user: name=temp state=absent remove=true</code>
Create a directory	<code>file: path=/opt/app state=directory mode='0755'</code>
Set ownership recursively	<code>file: path=/var/www recurse=true owner=www-data</code>
Create a symlink	<code>file: src=/opt/v2 dest=/opt/current state=link</code>
Copy a file	<code>copy: src=app.conf dest=/etc/app.conf mode='0644'</code>
Write inline content	<code>copy: content="line\n" dest=/etc/motd</code>
Ensure a line exists	<code>lineinfile: path=/etc/f regexp='^Key' line='Key=value'</code>
Remove a line	<code>lineinfile: path=/etc/f regexp='^Key' state=absent</code>

Table 6 — Quick reference for the essential modules.

This document describes Ansible Core behaviour as of version 2.14 and later. Always consult the official module documentation for the version in use, as parameters and defaults may change between releases.